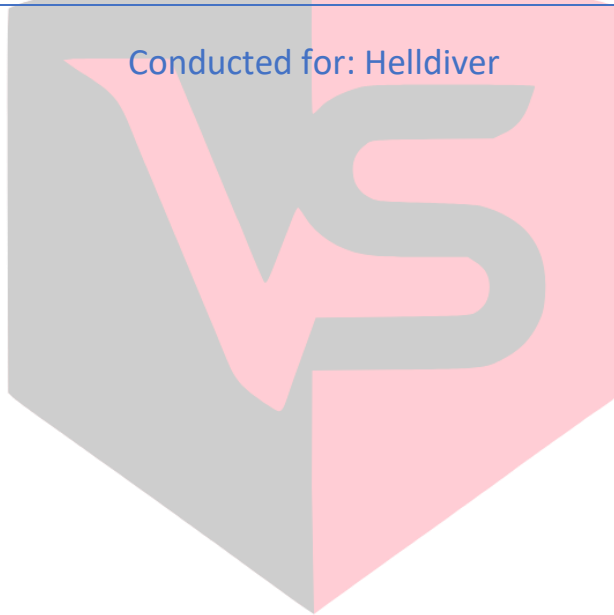




SMART CONTRACT AUDIT

Conducted for: Helldiver



VULSIGHT

Contents

Executive Summary:..... 3

Project Background:..... 3

Audit Scope:..... 3

Audit Summary:..... 3

Severity Definitions:..... 4

Technical Checks: 5

Code Quality and Documentation: 5

Audit Findings:..... 5

 Critical vulnerabilities: 5

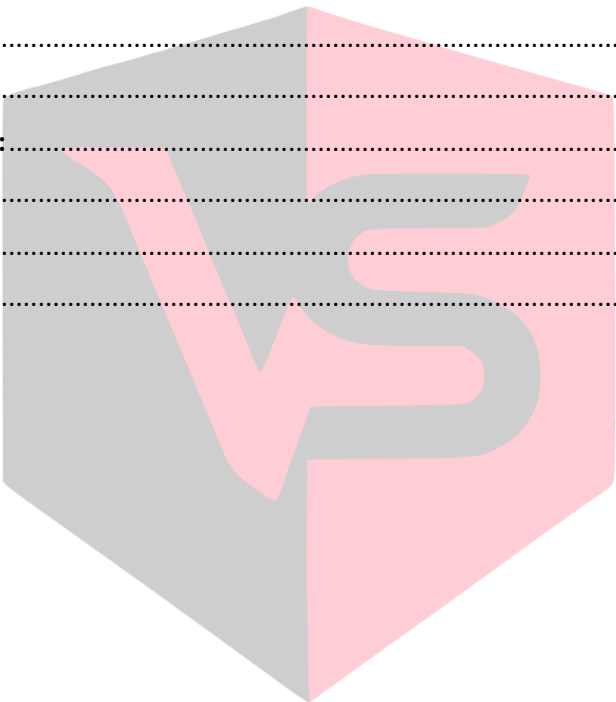
 High vulnerabilities: 5

 Medium vulnerabilities:..... 6

 Low vulnerabilities: 6

Disclaimer:..... 6

Conclusion: 7



VULSIGHT

Executive Summary:

The client contacted our Firm to conduct a smart contract audit on their Solidity based smart contracts. This audit followed a previous audit on a previous version of the contract deployed on the test net, which had vulnerabilities. The main purpose of the audit was to check whether the vulnerabilities found in the old contract are still present within the updated one. One of the contract was the basis for their ERC-20 Token, While the other one provides the functionalities to facilitate the presale of the tokens. In our comprehensive audit of the contracts, we carefully examined several critical aspects including functionality, security vulnerabilities, best practices, and smart contract efficiency. The contract meticulously initiates a token presale process, utilizing phased pricing to incentivize early participation while implementing features like automatic staking upon purchase. No significant vulnerabilities were discovered during this audit.

Project Background:

The smart contracts are written in solidity. The token contract is a simple ERC20 Contract with no additional functionalities. The presale contract has owner functionality within it. The contracts are deployed on the Binance mainnet.

Audit Scope:

Deployed Addresses	0x6b3722691e887BA67db49b5FAf1de509c34Dd3FA
Chain	BNB Chain
Language	Solidity
Audit Completion Date	5/5/24

Audit Summary:

- **Dynamic Pricing Mechanism:** The contract adopts a unique approach to pricing, utilising a predefined array of prices corresponding to distinct presale stages, influenced by the elapsed time since the presale launch. This strategy aims to reward early participants with lower prices.
- **Multi-currency Contributions:** Contributions can be made in the native currency (e.g., BNB/ETH) and the ERC20 stablecoins USDT and USDC, providing flexibility to participants. The conversion to the equivalent token amounts leverages a real-time price feed for the native currency, yet for

stablecoin contributions, it directly computes the number of tokens, showing an effective use of on-chain data and avoiding external dependency for stablecoin pricing.

- **Staking and Rewards:** The contract automatically stakes tokens purchased during the presale, with the APY and specific reward calculations encoded within.
- **Ownership and Administrative Functions:** The contract vests significant control in the owner for adjusting presale settings, token prices, and withdrawing funds. While such controls are common, they necessitate trust in the owner to act in the project's best interest. The presence of onlyOwner modifiers on sensitive functions mitigates unauthorized access risks.
- **Presale Status Management:** Dedicated functions to start and end the presale provide straightforward transitions between presale stages, including preparation for the open market.
- **Fund Withdrawal and Token Update Features:** Functionality allowing the owner to withdraw collected funds and update token addresses post-initial deployment introduces flexibility in managing and reallocating funds. While essential from an operational perspective, such features crucially require strong governance to prevent misuse.

Various tools such as Slither and different LLM scanners were used in the audit. Extensive manual code review was also done to ensure that any vulnerability if present could be detected in the audit.

We found:

- 0 Critical risk vulnerabilities
- 0 High risk vulnerabilities
- 0 Medium risk vulnerabilities
- 0 Low risk vulnerabilities

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to direct token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss

Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract
------------	--

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Passed
Logical Vulnerabilities	Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included two smart contracts, which are written in Solidity programming language. The code is well indented and clear in nature.

Audit Findings:

Critical vulnerabilities:

Zero critical vulnerabilities were found in the smart contract.

High vulnerabilities:

Zero high vulnerabilities were found in the smart contract.

Medium vulnerabilities:

Zero medium vulnerabilities were found in the smart contract.

Low vulnerabilities:

Zero low vulnerabilities were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.



Conclusion:

The smart contract was tested extensively both automatically and manually. No Vulnerabilities were discovered within it. It is recommended that additional security measures may be undertaken to ensure future security such that of a creation of a bug bounty program.

